



Chapter 8

フォーム処理の基本

Web サービスを行う上でフォーム入力は欠かせない機能です。フォームはHTMLで作りますが、ユーザからの入力データはPHPで処理します。フォームから送られてくる、GETリクエスト、POSTリクエストのデータを安全に処理するために必要になる基本的な知識を説明します。

-
- Section 8-1 HTTP の基礎知識
 - Section 8-2 フォーム入力処理の基本
 - Section 8-3 フォームの入力データのチェック
 - Section 8-4 隠しフィールドでPOSTする
 - Section 8-5 クーポンコードを使って割引率を決める
 - Section 8-6 フォームの作成と結果表示を同じファイルで行う
-

Section 8-1

HTTP の基礎知識

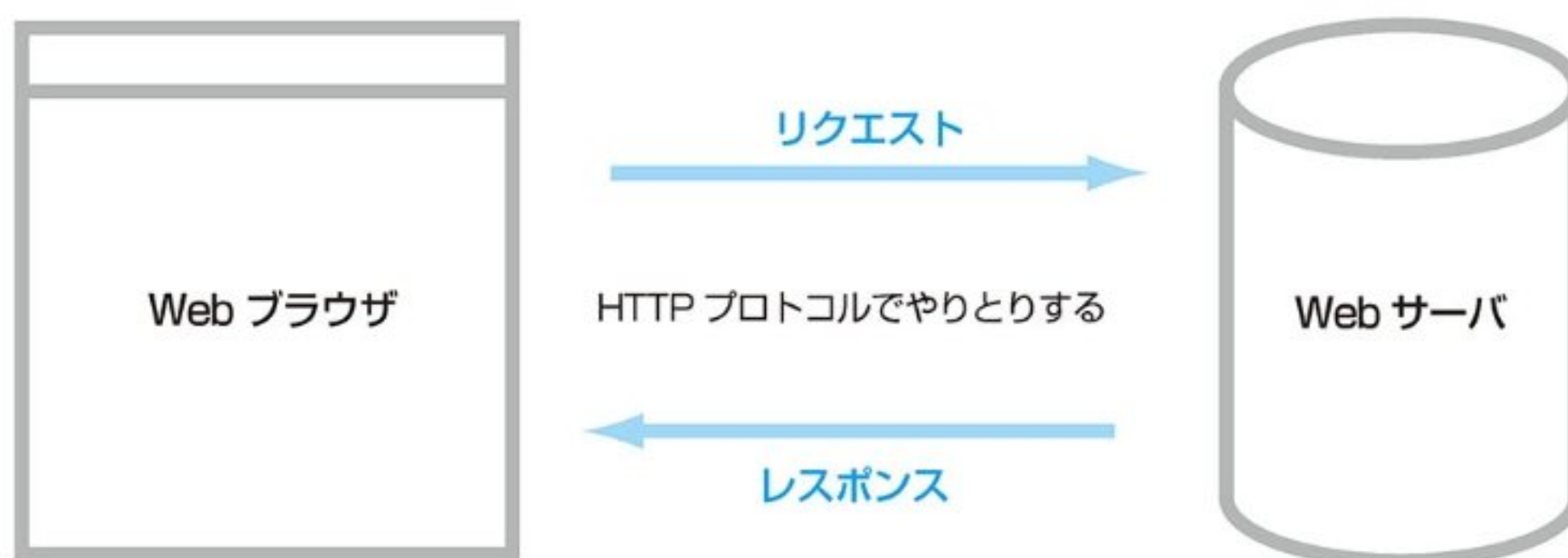
Web ページのフォームの入力処理を理解するには、Web サーバと Web ブラウザの間で行われるやり取り、すなわち HTTP (HyperText Transfer Protocol) について知っておくことが大事です。この節では HTTP の基本を簡単に説明します。

HTTP リクエストと HTTP レスponse

Web ページを開いたりフォーム入力を行ったりすると、Web サーバと Web ブラウザの間でデータのやり取りが行われます。このやり取りは、HTTP という仕様（プロトコル）に基づいて行われます。

Web ブラウザは、ブラウザの情報やフォーム入力データなどのデータのヘッダを添えて、開きたい Web ページのアドレスを Web サーバに要求します。この要求を「リクエスト」といいます。

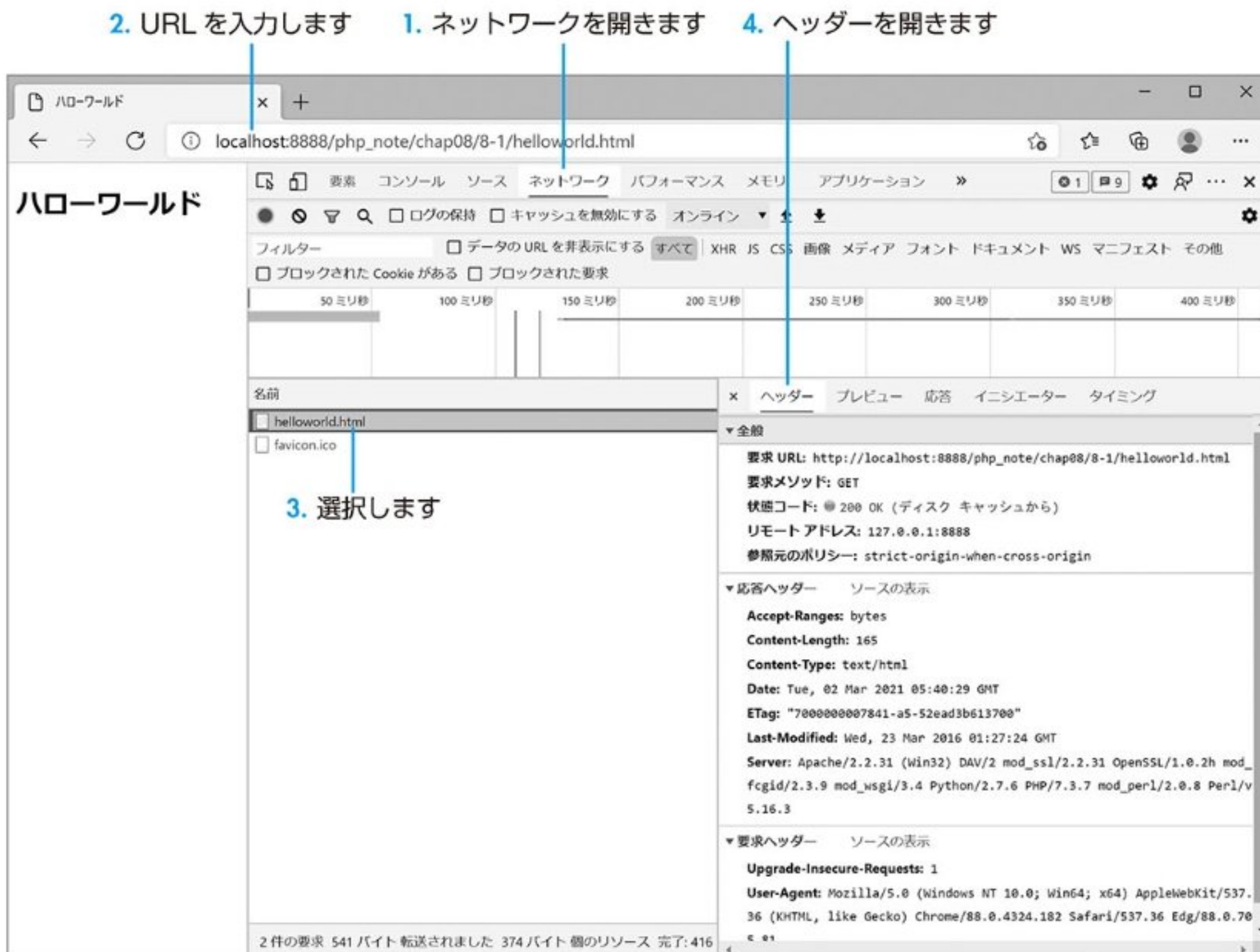
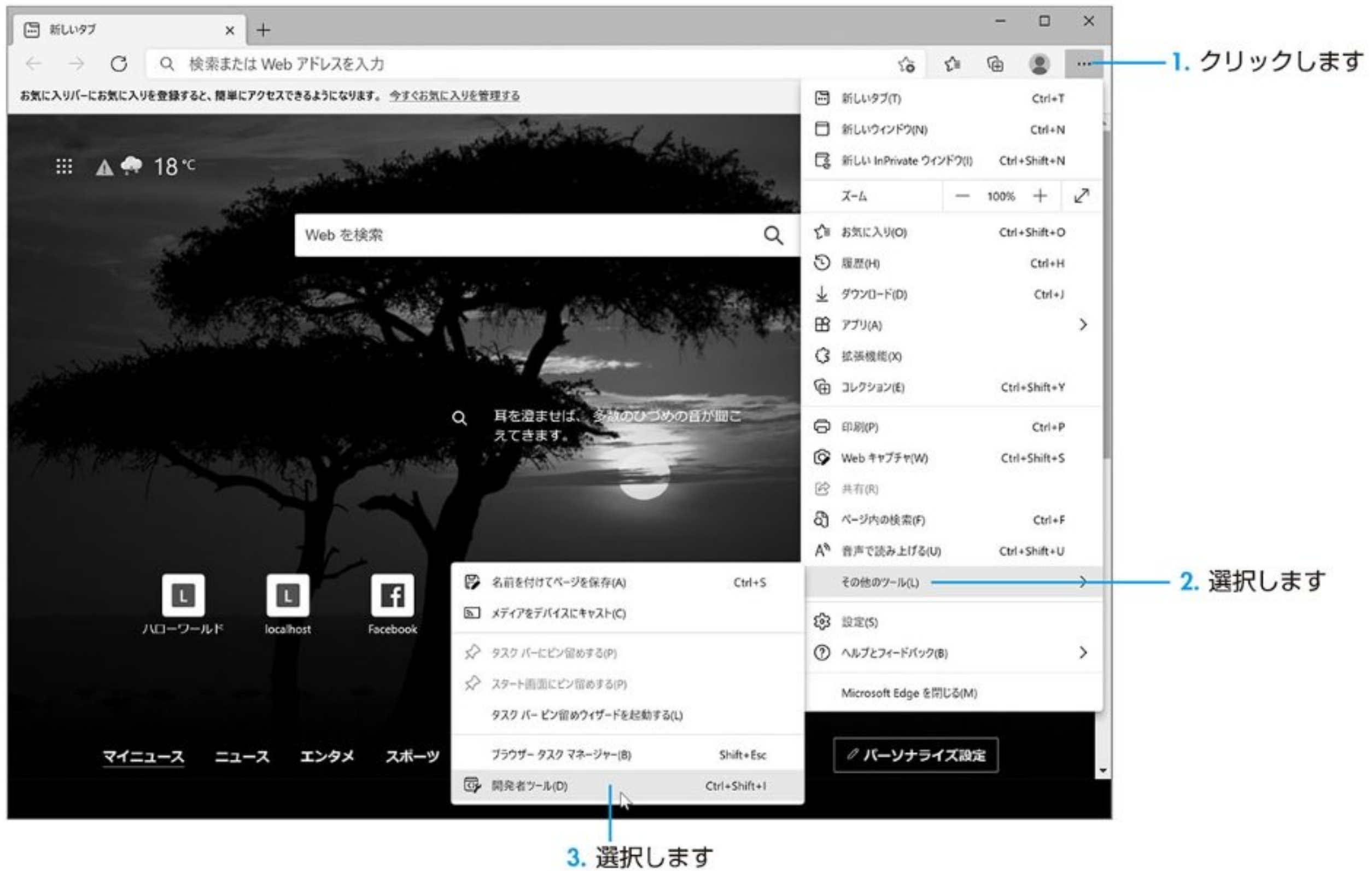
Web ページからの要求を受けた Web サーバは、サーバ情報や処理結果を示すエラーコードやメッセージのヘッダを添えて、Web ページのコンテンツを回答します。この回答を「レスポンス」といいます。



このやり取りの内容は Web ブラウザの開発ツール機能などで確認することができます。Windows 10 の Microsoft Edge の場合は、その他のツール>開発者ツールの「ネットワーク」の「ヘッダー」にリクエストとレスポンスの内容が表示されます。たとえば、「ハローワールド」と表示するだけの `helloWorld.html` を開いた場合を見てみましょう。

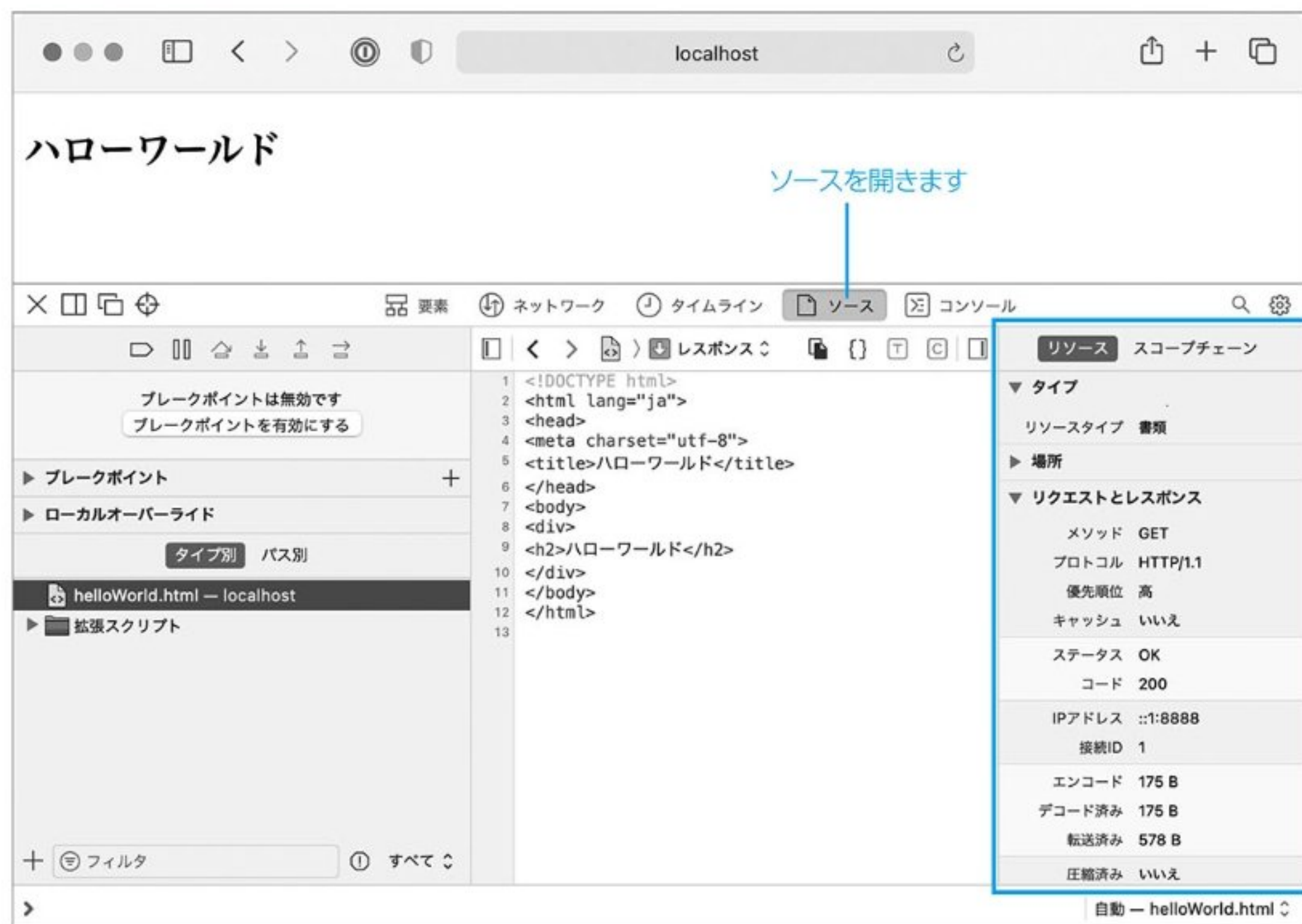


Windows 10 の Microsoft Edge で「その他のツール」>「開発者ツール」を選択する



macOS の Safari の場合は開発メニューの Web インスペクタの「リソース」にリクエストとレスポンスのヘッダの内容が表示されます。

macOS の Safari の Web インスペクタ



リクエストの内容

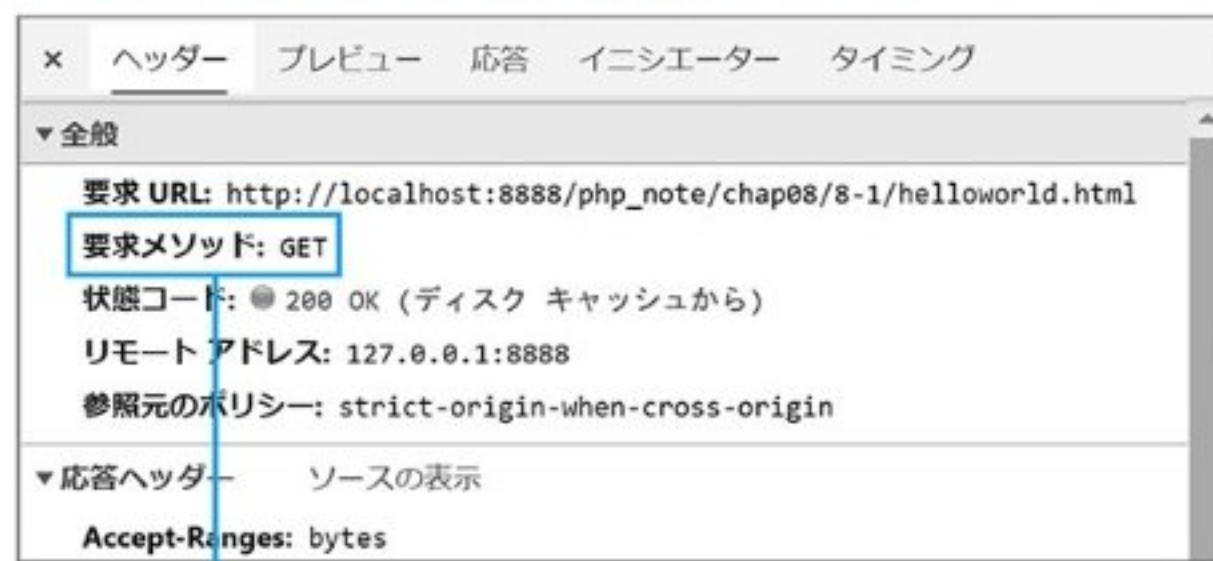
リクエストの先頭は次のような HTTP メソッドです。この行ではメソッドとドキュメントの URL、そしてプロトコルのバージョンを送っています。

GET /index.html HTTP/1.1

HTTP メソッド

先の開発ツールの図では、メソッドはリクエストの「要求メソッド」、「メソッド」の項目に GET と表示されています。HTTP メソッドでもっとよく用いられるのが GET と POST です。Web ブラウザのアドレスバーに URL を入力して Web ページを表示する場合にも GET でリクエストされます。

Windows 10 の Microsoft Edge の開発ツール



要求メソッド: GET

macOS の Safari の Web インスペクタ



メソッド GET

また、ボタンやフォーム入力の HTML で次のようなコードを目にしたことがあると思います。ここでは、method 属性で POST が指定されています。

<form method="POST" action="entry.php">

リクエストには、この後に User-Agent、Accept が続きます。User-Agent は Web ブラウザの情報、Accept は MIME タイプの指定です。

レスポンスの内容

Web サーバからのレスポンスの最初の 1 行は次に示すような HTTP ステータスです。この行ではプロトコルのバージョンに続いて、処理結果のコードとメッセージが書いてあります。コードの 200 番台は成功、300 番台はリダイレクト、400 番台はクライアントエラー、500 番台はサーバエラーを示します。

HTTP/1.1 200 OK — HTTP ステータス

レスポンスには、この後に Date、Server、Content-Type、Content-Length が続きます。Server は、Web サーバのソフトウェア情報です。

スーパーグローバル変数

PHP では Web サーバへのリクエストの情報、つまり、フォーム入力やクッキーの値、アップロードファイルの情報、サーバ側の環境変数、セッションの情報などを参照したり操作したりするためのスーパーグローバル変数をもっています。スーパーグローバル変数はどこからでも参照できる配列です。詳しくは改めて説明しますが、まとめると次のような配列があります。

変数名	内容
\$_GET	GET リクエスト (クエリ情報) のパラメータ。パラメータ名が配列のキーになる。
\$_POST	POST リクエストのパラメータ。パラメータ名が配列のキーになる。
\$_COOKIE	クッキーの値。クッキーの名前が配列のキーになる。
\$_SESSION	セッション変数。
\$_FILES	アップロードされたファイルの情報。
\$_SERVER	Web サーバに関する情報。
\$_ENV	サーバ側の環境変数。環境変数名が配列のキーになる。

上記以外に `$_REQUEST` があります。これは `$_GET`、`$_POST`、`$_COOKIE` をまとめた配列ですが、同名のキーが上書きされるといった理由から利用しない方がよいとされています。

GET と POST の違い

Web ブラウザから Web サーバへデータを送る HTTP メソッドでよく利用されるのが GET と POST です。GET と POST の違いは大きく 3 つあります。

1. GET はリクエストを URL に付けるのでブックマークできる

GET はパラメータを URL 形式にエンコードしたクエリ情報（クエリ文字列）を作って送信します。URL のアドレスの後に `?` を付けて、`キーと値のペアを続けた部分がクエリ文字列` です。複数のパラメータがある場合は `&` でつなぎます。

書式 クエリ文字列

URL? キー 1= 値 & キー 2= 値 & キー 3= 値

次の図で示す URL にはクエリ文字列の「`?goukei=2500&ninzu=3`」が付いています。このクエリ文字列には、`goukei` キーと `ninzu` キーの 2 つのパラメータが含まれています。

	パラメータ 1	パラメータ 2
<u>http://localhost:8888/ ... /warikan.php?goukei=2500&ninzu=3</u>		
URL	クエリ文字列	

このリクエストの内容は Web ブラウザのアドレスバーに表示されてしまうことから、これをブックマークできてしまいます。`ブックマークを呼び出すと GET リクエストを実行した場合と同じ結果になります`。これは場合によっては便利なこともありますが、本来は好ましくありません。

また、アドレスバーに表示されたリクエストをもとにして、`パラメータの値を変更したリクエストを再発行する`といったことも簡単にできます。

図で示すように GET リクエストは `<a>` タグを使って簡単に送信することができます。なお、フォームを使って GET リクエストを送信する方法は次節で説明します（[P.301](#)）。

1. リンクをクリックして GET リクエストを送ります

```
<a href="http://localhost:8888/php_note/chap08/8-1/get/warikan.php?goukei=2500&ninzu=3"> 割り勘を計算する </a>
```



3. アドレスバーにはクエリ文字列が表示されます



2. クエリの結果が表示されました

一方の POST はフォームのパラメータを URL に含めるのではなく、リクエストの本文に含めて送ります。したがって、GET のようにリクエスト内容を簡単に見られることがなく、ブックマークすることもできません。



セキュリティ対策

機密保持には暗号化通信を使う

アドレスバーに表示されないので、POST リクエストの内容が安全に保護されているということではありません。機密を保持した通信を行うには、SSL などの暗号化通信を利用してください。

2. GET で送信できるデータサイズに制限がある

POST のデータサイズは無制限であるのに対し、GET のクエリ情報には制限があります。利用する Web ブラウザ、サーバによってデータサイズの制限は異なりますが、URL のアドレスとの合計サイズでの上限があります。データサイズの制限がない実行環境であっても、極端に長い URL は動作が遅くなってしまうことがあります。

3. GET のレスポンスはキャッシュされるが POST はキャッシュされない

GET リクエストに対するレスポンスはキャッシュされます。したがって、同じ内容の GET リクエストは毎回同じ結果になります。つまり、いつも内容が変化しないレスポンスを得たい場合のリクエストに向いています。GET で毎回最新のレスポンスを得たい場合には、パラメータに時刻を付けることで毎回のリクエストを変更するといったテクニックが利用されていることがあります。これに対して POST リクエストに対するレスポンスはキャッシュされません。したがって、掲示板やショッピングカートの内容を表示したいといった場合には POST を使います。データベースの更新に GET を使ってはいけません。

Section 8-2

フォーム入力処理の基本

フォームにはテキストフィールドだけでなく、ラジオボタンやプルダウンメニューなど多くのタイプがありますが、基本的には同じように処理します。まずは簡単な例でフォーム処理の基本を見てみましょう。さらにセキュリティ対策として必要な HTML エスケープと文字エンコードのチェックするコードも紹介します。ユーザ定義関数 `es()` と `cken()` は次節からも利用します。

送信フォームを作る

HTML で送信フォームを作り、フォームのアクションで PHP プログラムを実行します。具体的には、フォームは `<form>` タグで囲み、その中に `<input>` タグでテキストフィールドや送信ボタンを作ります。`<form>` タグの `method` 属性で "POST" または "GET" を指定し、`action` 属性で実行する PHP ファイルを指定します。

使用するメソッド 実行する PHP ファイル

```
<form method="POST" action="calc.php">
```

この中にテキストフィールドや送信ボタンなどの UI 部品を指定します

```
</form>
```

テキストフィールドの値を POST メソッドで送信する

`calcForm.php` で作るフォームには、単価と個数のテキストフィールドがあり、「計算する」ボタンをクリックすると2つのテキストフィールドに入力された値を POST メソッドで `calc.php` に送ります。

`calc.php` では、フォームの値を取り出して「単価 × 個数」の計算結果を新しいページで表示します。

1. 単価と個数を入力します

2. クリックします

3. POST でリクエストします

4. 結果が表示されたページが開きます



php

POST メソッドのフォームを表示する

«sample» post/calcForm.php

```

01: <!DOCTYPE html>
02: <html lang="ja">
03: <head>
04: <meta charset="utf-8">
05: <title> フォーム入力処理の基本 (POST) </title>
06: <link href="../../../css/style.css" rel="stylesheet">
07: </head>
08: <body>
09: <div>
10:
11:   <form method="POST" action="calc.php">
12:     <ul>
13:       <li><label> 単価 : <input type="number" name="tanka" ></label></li>
14:       <li><label> 個数 : <input type="number" name="kosu" ></label></li>
15:       <li><input type="submit" value=" 計算する " ></li>
16:     </ul>
17:   </form>
18:
19: </div>
20: </body>
21: </html>

```

POST リクエストするフォームを作ります

PHP コードは含まれていないので、拡張子は html でも構いません

Part 3
Chapter 8
Chapter 9
Chapter 10
Chapter 11

テキストフィールドを作る

テキストフィールドは `<input>` タグで作り、`type` 属性で UI の形状を指定します。`type` 属性については改めて説明しますが、ここで重要なのは `name` 属性です。単価のテキストフィールドの `name` 属性には `"tanka"`、個数のテキストフィールドの `name` 属性には `"kosu"` と設定してあることに注目してください。

「単価：」、「個数：」のラベルとテキストフィールドを `<label> ~ </label>` で囲むとラベルをクリックしたときに該当するテキストフィールドにフォーカスが移動するようになります。

php

単価と個数を入力するテキストフィールドを作る

«sample» post/calcForm.php

```

13: <li><label> 単価 : <input type="number" name="tanka" ></label></li>
14: <li><label> 個数 : <input type="number" name="kosu" ></label></li>

```


送信ボタンを作る

送信ボタンも `<input>` タグで作ります。 `type` を "submit" にするとボタンになり、 `value` の値がボタン名として表示されます。

php 送信ボタンを作る

«sample» post/calcForm.php

```
15: <li><input type="submit" value=" 計算する " ></li>
```

ボタン名になります

POST された値を取り出す

フォームの送信ボタンがクリックされると `<form>` タグの `action` 属性に設定されていた `calc.php` が呼び出されます。 `calc.php` では、POST されたテキストフィールドの値を取り出して計算を行います。 `calc.php` のコードは次のような内容です。

なお、ここでは単価と個数に数値が入っていることを前提に処理しています。本来ならば値が入力されているか、数値計算ができる値であるかといったことをチェックする必要があります。入力データのチェックについては次節で説明します。

php POST メソッドを処理する PHP コード

«sample» post/calc.php

```
01: <!DOCTYPE html>
02: <html lang="ja">
03: <head>
04:   <meta charset="utf-8">
05:   <title> フォーム入力の値で計算する </title>
06:   <link href="../../../css/style.css" rel="stylesheet">
07: </head>
08: <body>
09: <div>
10: <?php
11:   // フォーム入力の値を得る（単価と個数）
12:   $tanka = $_POST["tanka"];
13:   $kosu = $_POST["kosu"];
14:   // 計算する
15:   $price = $tanka * $kosu;
16:   // 表示する（3桁位取り）
17:   $tanka = number_format($tanka);
18:   $price = number_format($price);
19:   echo " 単価 {$tanka} 円 × {$kosu} 個 は {$price} 円です。 "
20: ?>
21: </div>
22: </body>
23: </html>
```


POST された値を調べる

POST された値は `$_POST` グローバル変数に入ります。`$_POST` はフォームの `input` 項目の値の配列になります。入力された各値は、`name` 属性に付けた名前をキーにして配列 `$_POST` に保存されます。先の `calcForm.php` において、各 `<input>` タグの `name` 属性で「単価:」には `"tanka"`、「個数:」には `"kosu"` という名前を付けてあるので、単価は `$_POST["tanka"]`、個数は `$_POST["kosu"]` でアクセスできます。`number_format()` は数値を3桁位取りして表示する関数です (P.153)。

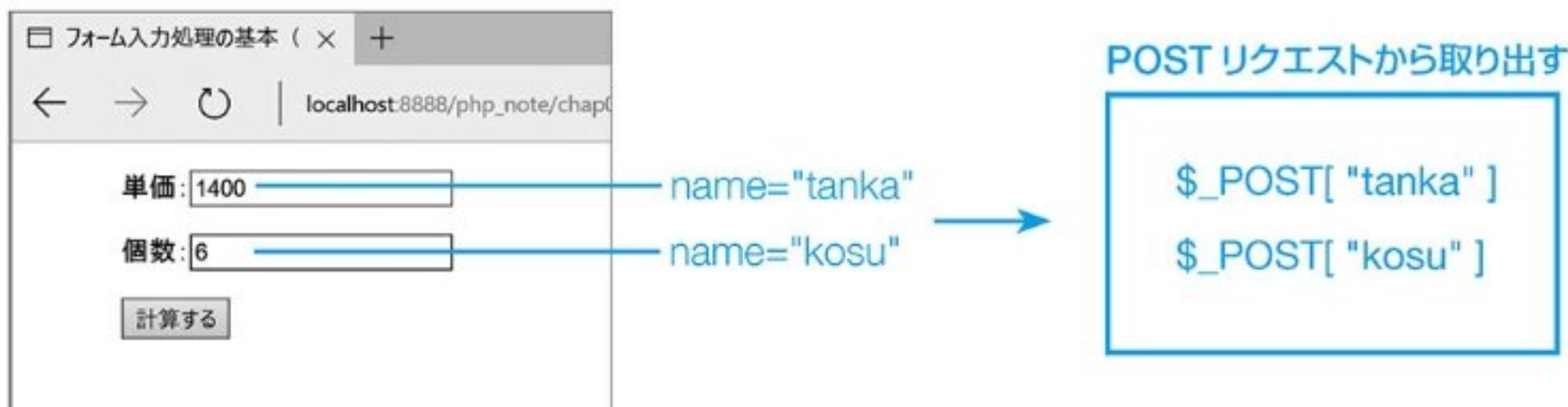
php POST された値を取り出す

«sample» post/calc.php

```

11: // フォーム入力の値を得る (単価と個数)
12: $tanka = $_POST["tanka"];
13: $kosu = $_POST["kosu"];

```



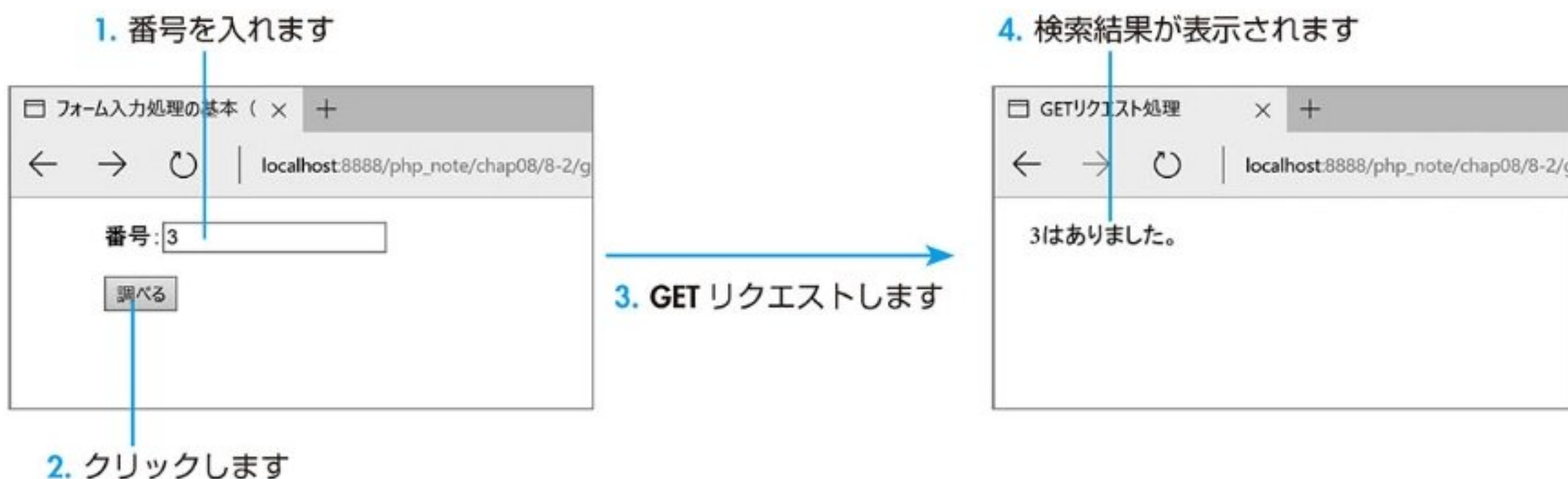
Part 3
Chapter 8
Chapter 9
Chapter 10
Chapter 11

フォームのボタンで GET メソッドで送信する場合

パラメータを URL に付加する形式の GET メソッドは、前節で説明したように `<a>` タグを使って簡単に送信することができますが、`<form>` タグの `method` 属性を `"GET"` にすれば POST リクエストと同じようにフォームを使って送ることもできます。

次の例で示すフォームには、番号を入力するテキストフィールドがあり、番号を入力して「調べる」ボタンをクリックすると入力された番号を GET メソッドで `checkNo.php` に送ります。

`checkNo.php` では、GET リクエストのクエリ文字列から番号を取り出して、登録番号の配列に入っているかどうかをチェックして、その結果を新しいページで表示します。





php GET メソッドのフォームを表示する

«sample» get/checkNoForm.php

```

01: <!DOCTYPE html>
02: <html lang="ja">
03: <head>
04: <meta charset="utf-8">
05: <title> フォーム入力処理の基本 (GET) </title>
06: <link href="../../../css/style.css" rel="stylesheet">
07: </head>
08: <body>
09: <div>
10:
11: <form method="GET" action="checkNo.php">
12:   <ul>
13:     <li><label> 番号 : <input type="number" name="no"></label></li>
14:     <li><input type="submit" value=" 調べる " ></li>
15:   </ul>
16: </form>
17:
18: </div>
19: </body>
20: </html>

```

PHP コードは含まれていないので、
拡張子は html でも構いません

GET メソッドでテキストフィールドの値を送ります

値を処理する PHP ファイルを指定します

GET された値を調べる

GET された値は `$_GET` グローバル変数に入ります。`$_POST` と同様に `$_GET` もフォームの `input` 項目の値の配列になります。入力された各値は、`name` 属性に付けた名前をキーにして配列 `$_GET` に保存されます。先の `checkNoForm.php` において、番号を入力する `<input>` タグの `name` には `"no"` という名前を付けてあるので、番号は `$_GET["no"]` でアクセスできます。配列の中に番号があるかどうかは `in_array()` で判断しています (P.229)。

php GET メソッドを処理する PHP コード

«sample» get/checkNo.php

```

01: <!DOCTYPE html>
02: <html lang="ja">
03: <head>
04:   <meta charset="utf-8">
05:   <title>GET リクエスト処理</title>
06:   <link href="../../../css/style.css" rel="stylesheet">
07: </head>
08: <body>
09: <div>

```



```

10: <?php
11: // GET リクエストのパラメータの値を受け取る
12: $no = $_GET["no"]; // GET された値を取り出します
13: // 番号リスト
14: $nolist = [3, 5, 7, 8, 9];
15: // 検索する
16: if (in_array($no, $nolist)){ // $no の値が配列 $nolist にあれば true を返します
17:     echo "{$no} がありました。";
18: } else {
19:     echo "{$no} が見つかりません。";
20: }
21: ?>
22: </div>
23: </body>
24: </html>

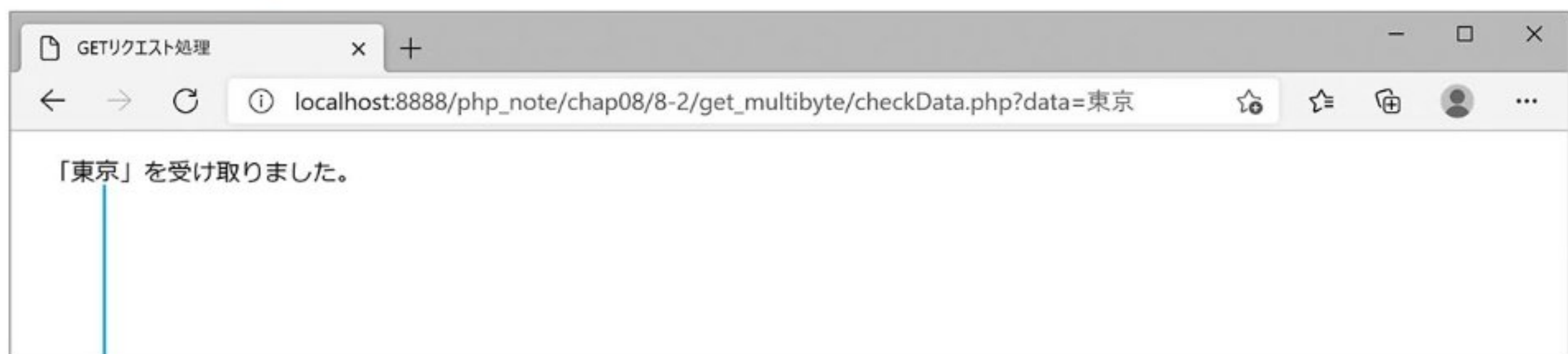
```

マルチバイト文字を URL エンコードする

GET リクエストのクエリ文字にマルチバイトが含まれている場合は、パラメータを URL エンコードしてから添付します。URL エンコードは `rawurlencode()` で行い、逆のデコードは `rawurldecode()` で行います。URL エンコードの必要がないブラウザもありますが、すべてのブラウザが対応しているわけではないので、この処理を行います。

なお、POST メソッドを使う場合は PHP がエンコードとデコードを自動で行ってくれるので、このような処理は必要ありません。

1. "東京" を URL エンコードしてクエリ文字列を作り、GET リクエストを送ります。



2. URL デコードして表示しています

この例では、\$url と \$data を使って {\$url}?data={\$data} の式でクエリ文字列を作っています。値をそのまま代入するとクエリ文字列は checkData.php?data=" 東京 " になりますが、data の値が " 東京 " というマルチバイト文字なので、式に代入する前に \$data を URL エンコードします。URL エンコードした結果で連結するとクエリ文字列は checkData.php?data=%E6%9D%B1%E4%BA%AC になります。

php マルチバイト文字を URL エンコードして GET リクエストする

«sample» get_multibyte/getRequest.php

```
01: <!DOCTYPE html>
02: <html lang="ja">
03: <head>
04: <meta charset="utf-8">
05: <title>URL エンコード (GET) </title>
06: <link href="../../../css/style.css" rel="stylesheet">
07: </head>
08: <body>
09: <div>
10: <?php
11: // URL エンコードする
12: $data = " 東京 ";
13: $data = rawurlencode($data);
14: // クエリ文字列のリンクを作る
15: $url = "checkData.php";
16: echo "<a href={$url}?data={$data}>", " 送信する ", "</a>";
17: ?>
18: </div>
19: </body>
20: </html>
```

クエリ文字列を作ります

GET リクエストを受け取って URL デコードする

受け取ったリクエストが URL エンコードされているものでも、\$_GET で値を取り出すのは同じです。data キーの値ならば、\$_GET["data"] で取り出すことができます。次の checkData.php では、取り出した値を rawurldecode() で URL デコードして元の文に戻して表示しています。

php GET リクエストを受け取り URL デコードする

«sample» get_multibyte/checkData.php

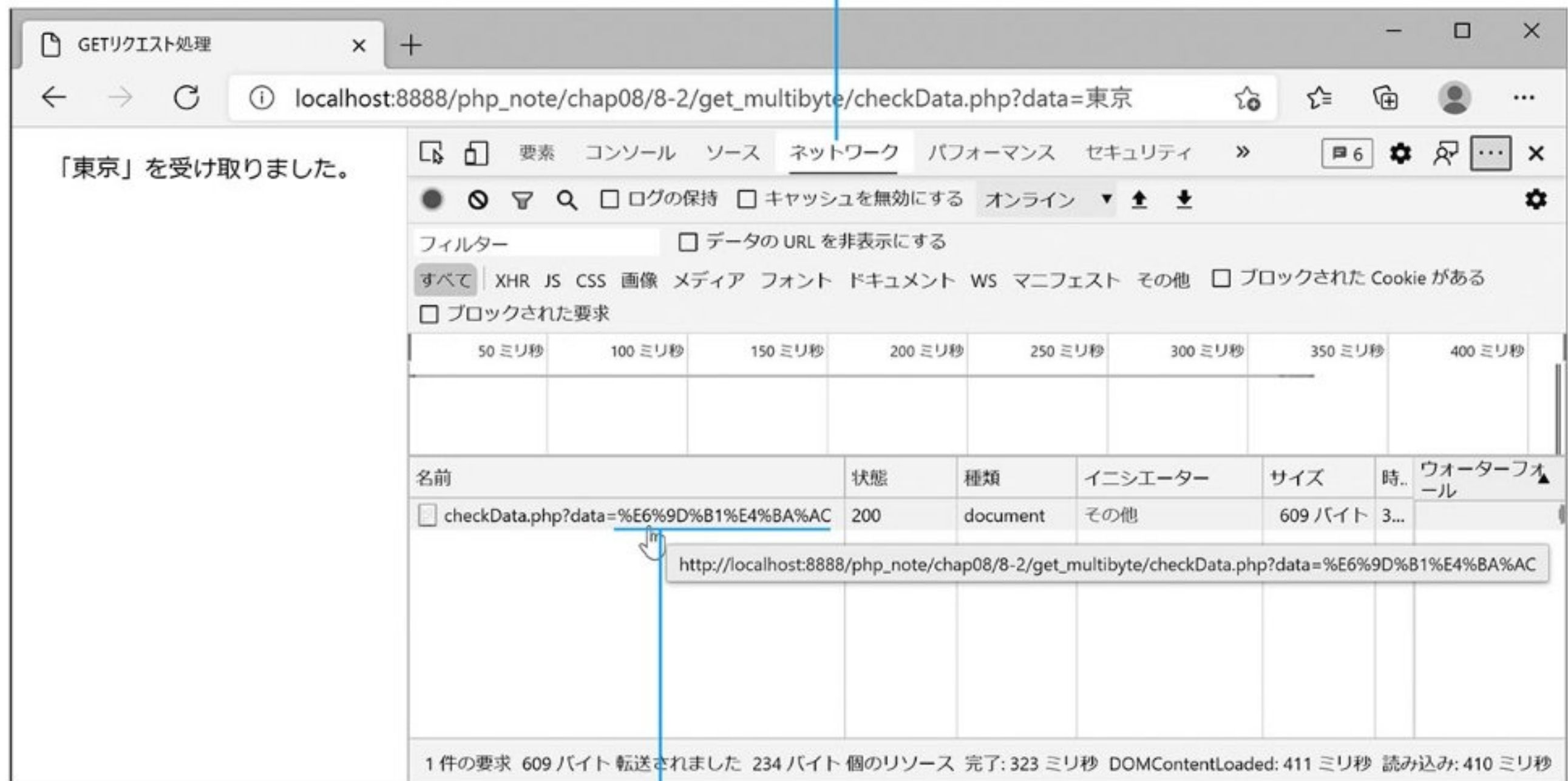
```
01: <!DOCTYPE html>
02: <html lang="ja">
03: <head>
04: <meta charset="utf-8">
05: <title>GET リクエスト処理</title>
06: <link href="../../../css/style.css" rel="stylesheet">
07: </head>
08: <body>
09: <div>
10: <?php
11: // GET リクエストのパラメータの値を受け取る
12: $data = $_GET["data"];
13: // URL デコードする
14: $data = rawurldecode($data);
15: echo "「{$data}」を受け取りました。";
16: ?>
17: </div>
18: </body>
19: </html>
```

読めるようにデコードします

Microsoft Edge ならばその他のツール>開発ツール、macOS Safari ならば開発メニュー> Web インスペクタで見ると URL の data の値がエンコードされているのを確認できます。

Microsoft Edge

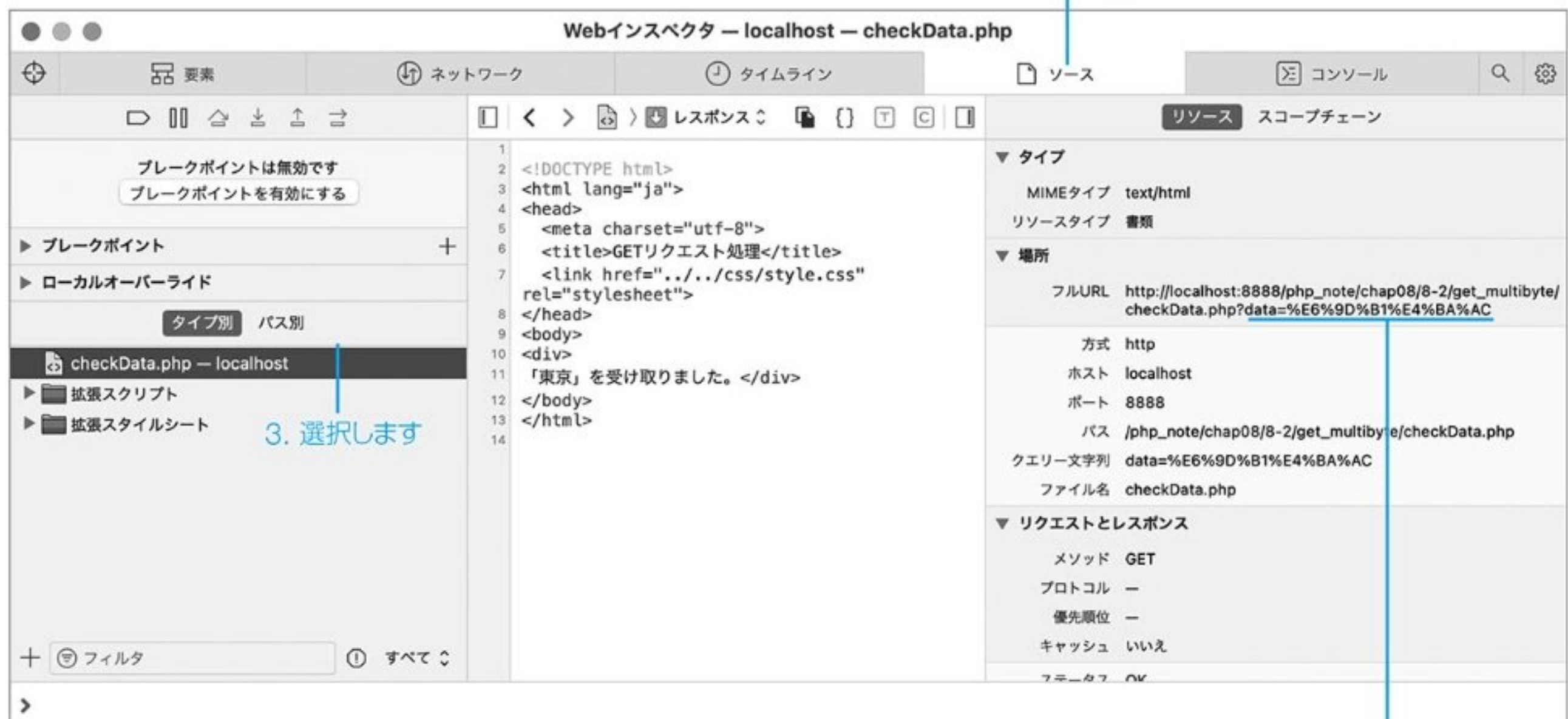
ネットワークを開きます



URL エンコーディングされています

macOS Safari

リソースを開きます



URL エンコーディングされています



セキュリティ対策

クロスサイトスクリプティング (XSS 対策)

GET リクエストはブラウザのアドレスバーの URL を書き替えるだけで改ざんできます。ユーザが送信内容を自由に入力できるフォーム入力は、簡単に HTML コードや JavaScript コードなどを送信できます。

このような改ざんを使ってブラウザで不正なスクリプトを実行させる攻撃手法を「クロスサイトスクリプティング (XSS)」と呼びます。ユーザに悪意がなくても、不用意に入力した <、> などの HTML コードで使う文字をそのまま表示するとブラウザでのレイアウトが崩れるといった不具合が出てしまいます。

不正な文字を HTML エスケープする

XSS に対抗する基本的な対策は、ユーザから受け取った値をブラウザに表示する前に、`htmlspecialchars()` を使用して値から不正な文字を HTML エスケープすることです。具体的には < > & " ' の 5 個の特殊文字を HTML エンティティ (<、>、&、"、') に変換します。HTML エンティティに変換された文字は、ブラウザでは元の文字で表示されます。

受け取ったデータを HTML エスケープする

先の例の `checkData.php` で XSS 対策を行うには、`rawurldecode()` で URL デコードした後で `htmlspecialchars()` を使って不正な文字を取り除く HTML エスケープを実行します。この処理はユーザから受け取ったデータをブラウザに表示する前に必ず行う必要があります。

`htmlspecialchars()` では変換する第 1 引数で文字列を指定し、第 2 引数には必ず `ENT_QUOTES` を指定します。なお、エンティティ変換せずに HTML コードを完全に取り除く方法もあります (P.162)。

書式 XSS 対策のための `htmlspecialchars()`

`htmlspecialchars (string: 値, flags:ENT_QUOTES, encoding:'UTF-8')`

php GET で受け取った値を URL デコードし、続いて HTML エスケープする

«sample» `xss_htmlspecialchars/checkData.php`

```

...
10:  <?php
11:    // GET リクエストのパラメータの値を受け取る
12:    $data = $_GET["data"];
13:    // URL デコードする
14:    $data = rawurldecode($data);
15:    // XSS 対策                                HTML エスケープします
16:    $data = htmlspecialchars(string:$data, flags:ENT_QUOTES, encoding:'UTF-8')
17:    echo "「{$data}」を受け取りました。";
18:  ?>
19:  ...

```

では実際に `checkData.php` に不正なコードの入った GET リクエストを送って試してみましょう。先の例の `getRequest.php` を実行して `checkData.php` を呼び出し、アドレスバーに表示されたリクエスト文字列の `data` の値を「<h1>Good
Bye!</h1>」に変更してページを読み込み直します。

すると htmlspecialchars() を通していない場合は HTML コードを表示して「Good Bye!」の文字が大きく改行されて表示されますが、htmlspecialchars() と通して表示すると HTML コードが「<h1>Good
Bye!</h1>」のようにエンティティされて変更した文字列がそのまま表示されます。

data=<h1>Good
Bye!</h1> に書き替えます

htmlspecialchars() を通さなかった場合
HTML コードがそのまま実行されます。

「**<h1>Good
Bye!</h1>**」を受け取りました。

危険!

htmlspecialchars() を通した場合
HTML コードがエンティティになって表示されます。

「<h1>Good
Bye!</h1>」を受け取りました。

htmlspecialchars() を便利に使うためのユーザ定義関数 es()

ユーザからのデータをブラウザに表示する前に htmlspecialchars() を通して HTML エスケープを行うことが必須となりますが、この処理を行うために array_map() をうまく利用したユーザ定義関数を作っておくと便利です (array_map() P.238)。

次の util.php に定義している es() では引数 \$data を is_array() でチェックして、\$data の値が配列ではない場合はそのまま htmlspecialchars() を実行し、配列ならば array_map() を使って値を順に __METHOD__ つまり es() で処理する式を return します。これは再帰呼び出しという手法です。こうすることで、es() は引数が文字列でも配列でも htmlspecialchars() で処理できる関数になります。

php 引数に対して htmlspecialchars() を実行する es()

«sample» lib_es/lib/util.php

```

01: <?php
02: // XSS 対策のための HTML エスケープ
03: function es(array|string $data, string $charset='UTF-8'):mixed {
04:     // $data が配列のとき
05:     if (is_array($data)){
06:         // 再帰呼び出し
07:         return array_map(__METHOD__, $data); // 配列の場合は、値を1つずつ引数にして、
08:     } else { // HTML エスケープを行う // 再帰呼び出しします
09:         return htmlspecialchars(string:$data, flags:ENT_QUOTES, encoding:$charset);
10:     }
11: }
12: }

```

NOTE

__METHOD__ を利用した再帰呼び出し

array_map() でコールしている __METHOD__ は、現在実行中のメソッド自身を指す特殊な定数（マジック定数）です。ここでは es() を指すので、es() の中で es() を使っていることになります。この手法を再帰呼び出しと言います。

es() を試してみる

では、この es() を試してみましよう。es() は lib フォルダの中の util.php に定義してある関数なので、require_once("lib/util.php") で読み込んで利用します。次の例では \$myCode には 1 個の文字列が入っており、\$myArray は複数の文字列が入っている配列です。

php es() をテストする

«sample» lib_es/esSample.php

```

01: <!DOCTYPE html>
02: <html lang="ja">
03: <head>
04:     <meta charset="utf-8">
05:     <title>XSS 対策 es()</title>
06:     <link href="../../../css/style.css" rel="stylesheet">
07: </head>
08: <body>
09: <div>
10: <pre>
11: <?php
12: // util.php を読み込む
13: require_once("lib/util.php");
14: // HTML タグの入ったデータを用意する
15: $myCode = "<h2>テスト 1</h2>";
16: $myArray = ["a"=>"<p>赤</p>", "b"=>"<script>alert('hello')</script>"];
17: // es() で HTML エスケープして表示する
18: echo '$myCode の値: ', es($myCode); // 変数 $mycode の値を HTML エスケープします
19: echo PHP_EOL . PHP_EOL ;
20: echo '$myArray の値: ';
21: print_r(es($myArray));
22: ?> // 配列 $myArray に入っているすべての値を HTML エスケープします
23: </pre>

```



```
24: </div>
25: </body>
26: </html>
```

出力

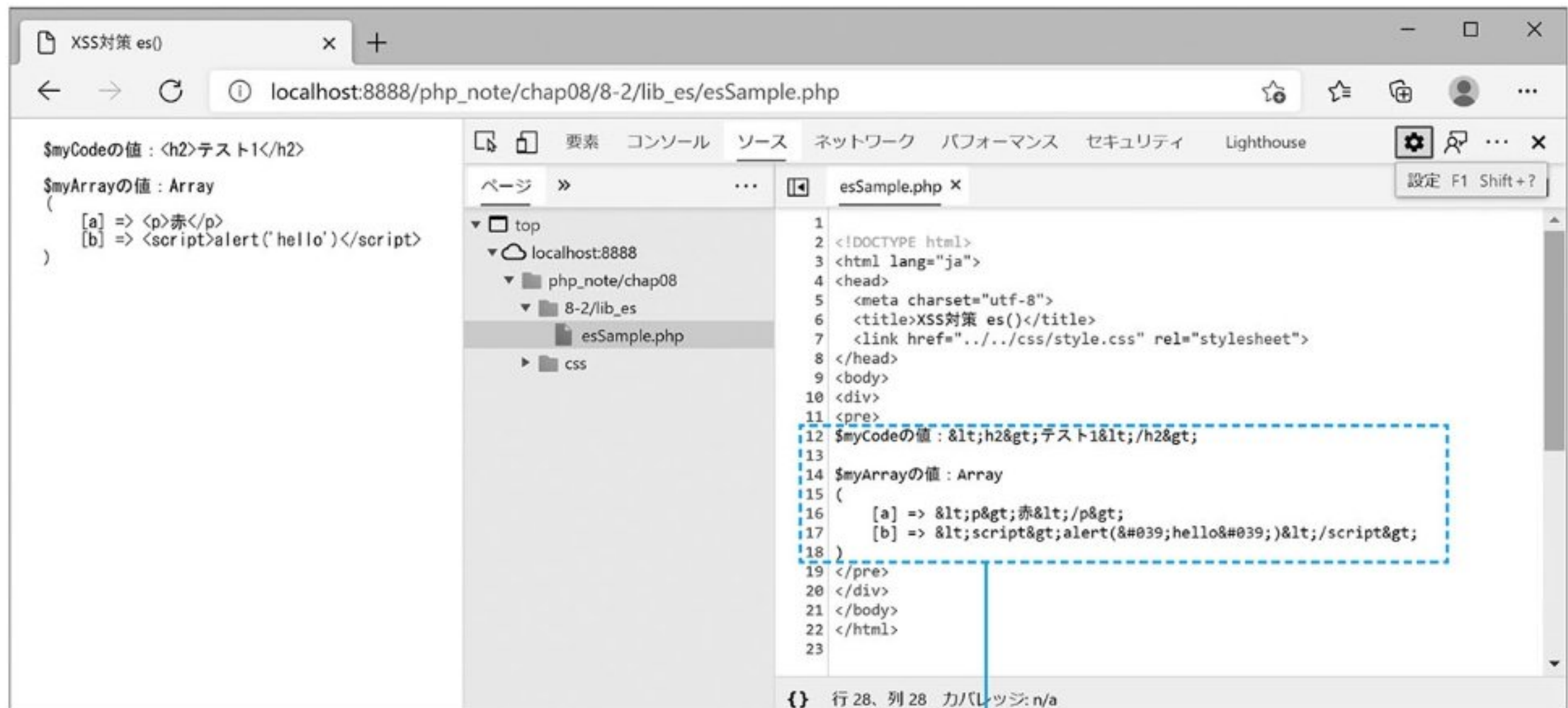
\$myCode の値 : <h2>テスト1</h2>

\$myArray の値 : Array

```
(
  [a] => &lt;p&gt;赤&lt;/p&gt;
  [b] => &lt;script&gt;alert(&#039;hello&#039;)&lt;/script&gt;
)
```

HTML コードが安全に置換されています

プログラムを実行するとブラウザには変数と配列の値がそのまま表示されますが、実際に出力された結果を確認すると値に含まれている特殊文字がエンティティ変換されています。



ブラウザでは入力データのままだに見えても、実際の値はエンティティ変換されています



**セキュリティ対策 不正なエンコーディングによる攻撃**

POST や GET で送られてくるデータの文字エンコードのチェックを行っておくことも大事です。文字エンコードのチェックは `mb_check_encoding()` で行うことができます。

文字エンコードを行うユーザ定義関数 `cken()`

`mb_check_encoding()` を使って文字エンコードのチェックを効率よく行う `cken()` を、先の `es()` と同様に `util.php` に定義しておきましょう。この関数は `$_GET`、`$_POST`、`$_SESSION` などの配列に含まれている値のエンコードをチェックすることを前提にしています。

`foreach` 文で配列から値を順に `$value` に取り出し、もし入っていた値が配列ならば `implode()` を使って値を1個の文字列に連結しておいてから (P.207)、`mb_check_encoding()` で文字エンコードをチェックします (1 階層の多次元配列までに対応)。文字エンコードが一致しないときは `$result` に `false` を代入して `foreach` 文の繰り返しをブレイクします。

最終的に `$result` が初期値の `true` のままであれば文字エンコードは正しく、途中で `false` が代入されていれば文字エンコードは一致していないことになります。

php 配列の文字エンコードのチェックを行う

«sample» lib_cken/lib/util.php

```

14: // 配列の文字エンコードのチェックを行う
15: function cken(array $data): bool{
16:     $result = true;
17:     foreach ($data as $key => $value) {
18:         if (is_array($value)){
19:             // 含まれている値が配列のとき文字列に連結する
20:             $value = implode("", $value); // 配列に入っている値を連結したストリングにしてチェックします
21:         }
22:         if (!mb_check_encoding($value)){
23:             // 文字エンコードが一致しないとき
24:             $result = false;
25:             // foreach での走査をブレイクする
26:             break;
27:         }
28:     }
29:     return $result;
30: }
31: // ?>

```

`cken()` をテストする

それでは `cken()` をテストしてみましょう。ここでは、利用環境が UTF-8 のときに Shift-JIS の文字列が入っている配列をテストします。`mb_convert_encoding()` で Shift-JIS に変換した文字列 `$sjis_string` を作成し、これを配列に入れて `cken()` でチェックします。なお、現在の利用環境の内部文字エンコードは `mb_internal_encoding()` で調べることができます。

php cken() をテストする

«sample» lib_cken/ckenSample.php

```
11: <?php
12: // util.php を読み込む
13: require_once("lib/util.php");
14: // Shift-JIS のデータを用意する      テスト用に Shift-JIS に変換します
15: $utf8_string = "こんにちは。";
16: $sjis_string = mb_convert_encoding($utf8_string, 'Shift-JIS');
17: // 内部エンコーディングを調べる
18: $encoding = mb_internal_encoding();
19: // cken() でチェックする
20: if (cken([$sjis_string])) {
21:     echo ' 配列の値は、', $encoding, ' です。';
22: } else {
23:     echo ' 配列の値は、', $encoding, ' ではありません。';
24: }
25: ?>
```



読み込んでいる CSS コード style.css

最後にこのサンプルで適用している CSS ファイル `style.css` のコードを示しておきます。この CSS ファイルはこの後の節のサンプルでも利用します。したがって、個々のサンプルでは使用していない属性の指定なども含まれています。

css 共通のスタイルシート

«sample» css/style.css

```
01: @charset "UTF-8";
02:
03: div{
04:     margin: 1em;
05: }
06:
07: li {
08:     list-style-type: none;
09:     margin-bottom: 1em;
10: }
11:
12: ol > li {
13:     list-style-type: decimal;
14:     margin-bottom: 0;
15: }
16:
17: a{
18:     color: #5e78c1;
19:     text-decoration: none;
20: }
21: a:hover{
22:     color: #b04188;
23:     text-decoration: underline;
24: }
25:
26: .error {
27:     color: #FF0000;
28: }
```

Part 3
Chapter

8

Chapter

9

Chapter

10

Chapter

11